

Adaptive Multi-Modal Edge Perception for Autonomous Mobile Robots

Research report - Adaptive Edge AI Perception Engine

1. Abstract

Mobile robots cannot run every state-of-the-art perception model simultaneously on embedded hardware. We present an adaptive perception engine that selects, per frame, which models run - trading perception breadth against compute as a function of fused scene risk and live resource telemetry. On a 120-frame synthetic patrol mission with two scripted events, the adaptive scheduler uses 39% fewer model-cost proxy units than a fixed heavyweight pipeline and achieves 4.9x its mean FPS, while running the full model suite on 87% of genuinely high-risk frames (fixed light baseline: 0%).

Validation status: these scheduler results use deterministic synthetic scenes and latency-simulating mock models; they evaluate scheduler behaviour, not real model accuracy or Jetson performance. Real-model execution is supported but is not part of CI. Jetson TensorRT, Gazebo end-to-end runtime, metric depth and labelled-dataset accuracy remain deployment work.

2. Problem

Edge GPUs (e.g. Jetson Orin Nano, ~40 TOPS at 7-15 W) cannot sustain a heavy detector + segmentation + depth + pose at camera rate. Fixed lightweight pipelines can omit useful safety context such as pose, segmentation and relative-depth cues exactly when it matters. Perception should therefore be a scheduled resource, not a static graph.

3. Architecture

Camera -> Adaptive Scheduler -> Model Plan -> Multi-Model Inference -> Perception Fusion + Confidence -> ROS2. The scheduler is a hysteresis state machine over four modes (PATROL / ALERT / ENGAGED / DEGRADED) driven by two signals: fused scene risk and compute headroom (FPS, GPU utilization, power).

4. Models

Task	Model	Cost tier	Role
Detect	YOLOv8 n/s/l	1/2/4	object hypotheses
Segment	YOLOv8n-seg	3	instance masks
Rel. depth	MiDaS small	3	relative depth cue
Pose	YOLOv8n-pose	2	human pose / activity cue
VLM	LLaVA / VILA (on-demand)	8	optional scene narration

5. Optimization pipeline

PyTorch -> ONNX (fixed shape, batch 1, simplified) -> TensorRT FP16/INT8, engines built on-device with entropy calibration on 500-1000 mission frames. Measured on this development machine (CPU; TensorRT column to be completed on Jetson):

Format	Device/EP	FPS	p50 (ms)	p95 (ms)
PyTorch	cpu	7.3	132.7	171.6
ONNX Runtime	CPU	7.1	129.3	161.4

6. Adaptive scheduler

Escalation is immediate (safety-first); de-escalation requires 20 consecutive calm frames (10 when compute-starved). Documented failure mode discovered during development: blocking de-escalation while compute-starved deadlocks the system in ENGAGED - the slow plan lowers FPS, which reads as starvation, which prevented calm frames from accumulating. Fix: starvation accelerates load-shedding instead of blocking it.

7. Experiments and results

7.1 Experiment 1 - format optimization

See Section 5 table. To complete on Jetson: TensorRT FP16 and INT8 rows plus power (W) per format.

7.2 Experiment 2 - adaptive vs fixed scheduler

120-frame mission, events at [[30, 41], [80, 91]], backend=mock (deterministic, hardware-independent). Cost proxy units sum manually assigned per-model scores over models that actually executed - not measured energy.

System	Mean FPS	FPS σ	Lat mean (ms)	Lat p95 (ms)	Cost proxy units	Event-window coverage	High-risk coverage	Switches
fixed_heavy	6.7	0.12	148.7	169.1	1440	1.0	1.0	0
fixed_light	89.8	2.72	11.0	13.5	120	0.0	0.0	0
adaptive	33.0	35.08	92.5	169.0	885	0.64	0.87	5

The adaptive scheduler runs the full suite on 87% of high-risk frames while spending 39% fewer cost units than the fixed heavyweight baseline. Event-window coverage (64%) is lower by design: the scripted event includes an approach ramp whose early frames carry genuinely low risk, where ALERT is the correct response. High FPS variance is the signature of mode switching, not instability.

7.3 Experiment 3 - multi-model understanding demo

Before (detection only): person detected; chair detected; chair detected; backpack detected ...

Backend provenance: yolo_nano:mock, yolo_large:mock, yolo_nano_seg:mock, midas_small:mock, yolo_pose:mock, vlm_scene:mock

object	rel depth	activity	risk %	confidence
person	0.328	upright	40	0.81
chair	0.690	n/a	14	0.72
chair	0.493	n/a	20	0.72
backpack	0.598	n/a	19	0.71

Relative depth uses 0=near and 1=far and is not metric. VLM narration: A person is visible nearby and may partially block the planned path.

7.4 Tier agreement with large-model pseudo-labels

7.5 Experiment 4 - ROS2 end-to-end latency

The perception node stamps end-to-end latency (camera header stamp -> publish) into /perception/status on every frame. To run: launch perception_tb3.launch.py and `ros2 topic echo /perception/status`.

8. ROS2 deployment

Topics: /perception/objects (vision_msgs/Detection2DArray; class and score only), /perception/scene (JSON including relative depth), /perception/risk, /perception/semantic_image_grid (16x12 image-space class grid; not a metric occupancy map), and /perception/status (mode, models, backend provenance, FPS and validated latency). Relative depth is deliberately not written to pose.z.

9. Future work

Learned scheduler (contextual bandit over model plans, reward = coverage - lambda*energy) against the hand-tuned state machine; continual learning loop (low-confidence crops auto-labeled by the large model, nano fine-tuned and re-exported to TensorRT); VLM tier served by TensorRT-LLM on Jetson.

10. Validation status

Environment	Status
Python 3.10-3.12, mock backend	CI tested
Lite Docker image	CI tested
ROS 2 node logic (mocked rclpy)	CI tested
ROS 2 Humble colcon build	CI tested in Humble container
Real-model path	supported; not part of CI
Gazebo edge_bot runtime	launch + URDF provided; runtime validation recommended
Jetson Orin TensorRT / INT8	scripts provided; device validation required
Metric depth	not implemented (relative only)

11. References

- Jocher et al., Ultralytics YOLOv8, <https://github.com/ultralytics/ultralytics> (AGPL-3.0).
- Ranftl et al., Towards Robust Monocular Depth Estimation, TPAMI 2022 (MiDaS), <https://github.com/isl-org/MiDaS>.
- Bhat et al., ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth, 2023 - recommended for metric range.
- NVIDIA TensorRT documentation, <https://docs.nvidia.com/deeplearning/tensorrt/>.
- Macenski et al., Robot Operating System 2: Design, architecture, and uses in the wild, Science Robotics 7(66), 2022.
- Zhang et al., ByteTrack: Multi-Object Tracking by Associating Every Detection Box, ECCV 2022 - planned tracker upgrade.
- Guo et al., On Calibration of Modern Neural Networks, ICML 2017 - calibration metrics for future uncertainty work.

Reproduce every table: `python -m benchmark.latency_test / format_test / accuracy_test / understanding_demo`. All figures in this report are generated from the committed benchmark JSON.